

CyberSOC Platform

Kubernetes Production Deployment Guide

Phase 2: Go-Live Roadmap Execution

Document Type	Technical Deployment Guide
Version	1.0.0 - Production Ready
Classification	Internal Technical
Date	2026-08-24
Platform	CyberSOC AI-Native SOC OS
Target Environment	Kubernetes Production Cluster

Table of Contents

1. Executive Summary — Overview and deployment objectives
2. Architecture Overview — Production architecture design
3. Prerequisites & Requirements — Infrastructure and tooling requirements
4. Cluster Configuration — Production cluster setup specifications
5. Helm Chart Structure — Complete Helm chart organization
6. Core Manifests — Deployment, Service, ConfigMap, Secret definitions
7. Database Deployment — PostgreSQL, Redis, Elasticsearch deployment
8. Service Mesh Integration — Istio configuration and traffic management
9. Security Hardening — Pod security, network policies, RBAC
10. Deployment Procedures — Step-by-step deployment workflow
11. Scaling Strategies — Horizontal and vertical scaling configurations
12. Monitoring & Observability — Prometheus, Grafana, logging stack
13. Disaster Recovery — Backup, restore, and failover procedures
14. Troubleshooting Guide — Common issues and resolution steps

1. Executive Summary

This Kubernetes Production Deployment Guide provides comprehensive technical documentation for deploying the CyberSOC Platform (AI-Native Security Operations Center Operating System) into production Kubernetes environments. The guide encompasses complete infrastructure-as-code artifacts including production-ready Kubernetes manifests, Helm charts for package management, detailed deployment procedures, and operational runbooks for maintaining the platform at scale.

1.1 Document Scope and Objectives

The primary objective of this deployment guide is to enable DevOps and Site Reliability Engineering teams to deploy, configure, and maintain the CyberSOC Platform in production Kubernetes clusters with confidence and repeatability. This document addresses the complete lifecycle of containerized deployment operations from initial cluster provisioning through ongoing operational maintenance, scaling events, and disaster recovery scenarios. The guide is structured to support both greenfield deployments in new Kubernetes environments as well as migration paths from existing staging or development deployments to full production readiness.

1.2 Target Audience

This technical documentation is specifically designed for platform engineers, DevOps practitioners, site reliability engineers, and security architects who are responsible for deploying and operating cloud-native security platforms. Readers should possess intermediate to advanced knowledge of Kubernetes orchestration concepts including pod lifecycles, controller patterns, service discovery mechanisms, and network policy implementation. Familiarity with Helm package management, container security best practices, and observability stack integration is assumed throughout the technical procedures outlined in subsequent sections.

1.3 Deployment Targets and Success Criteria

Metric	Target Value	Measurement Method
Deployment Time	< 45 minutes	Automated pipeline timing
Availability SLA	99.95% uptime	Prometheus SLO monitoring
Recovery Time Objective (RTO)	< 15 minutes	Failover drill execution
Recovery Point Objective (RPO)	< 5 minutes	Backup verification testing
Scale-out Latency	< 90 seconds	HPA metric observation
Security Compliance	SOC 2 Type II	Annual audit certification

Table 1.1: Production Deployment Success Criteria

2. Architecture Overview

The CyberSOC Platform employs a microservices architecture designed specifically for Kubernetes-native deployment, leveraging cloud-native patterns for scalability, resilience, and operational excellence. The production architecture implements defense-in-depth security principles at every layer, from container image integrity verification through network segmentation and zero-trust service-to-service authentication. This section provides a comprehensive overview of the architectural components, their interactions, and the design decisions that inform the deployment configuration presented throughout this guide.

2.1 High-Level System Architecture

The production deployment organizes CyberSOC platform components into distinct namespaces following Kubernetes best practices for multi-tenant isolation and operational boundary delineation. The core platform services reside within the `cybersoc-system` namespace, providing foundational capabilities including API gateway routing, authentication and authorization services, configuration management, and centralized logging aggregation. Application-layer services including threat detection engines, incident response workflows, compliance monitoring modules, and user interface components deploy into the `cybersoc-workloads` namespace, enabling independent scaling and rollout strategies without impacting critical system infrastructure.

2.2 Component Inventory and Responsibilities

Component	Namespace	Replicas	Resource Class	Description
API Gateway	cybersoc-system	3	cpu: 500m, mem: 512Mi	Edge routing, rate limiting, TLS termination
Auth Service	cybersoc-system	2	cpu: 300m, mem: 256Mi	OAuth/OIDC, JWT issuance, session mgmt
Threat Engine	cybersoc-workloads	5	cpu: 2000m, mem: 4Gi	AI/ML threat detection pipeline
SIEM Core	cybersoc-workloads	3	cpu: 1500m, mem: 3Gi	Event correlation, alert generation
Case Mgmt	cybersoc-workloads	2	cpu: 500m, mem: 1Gi	Incident tracking, workflow engine
UI Frontend	cybersoc-workloads	3	cpu: 200m, mem: 256Mi	React-based admin console
PostgreSQL	cybersoc-data	3	cpu: 1000m, mem: 4Gi	Primary relational data store
Redis Cluster	cybersoc-data	6	cpu: 500m, mem: 1Gi	Caching, session state, pub/sub
Elasticsearch	cybersoc-data	5	cpu: 2000m, mem: 8Gi	Log aggregation, full-text search

Table 2.1: Production Component Inventory

2.3 Network Topology and Traffic Flow

Production network architecture implements a layered security model with clearly defined trust boundaries and encrypted communication channels between all components. External traffic enters through cloud provider load balancers (AWS ALB/NLB or GCP Load Balancer) which terminate TLS connections using certificates managed via cert-manager with Let's Encrypt or enterprise PKI integration. The API Gateway receives decrypted HTTP/HTTPS traffic and applies layer 7 routing rules, authentication validation, and rate limiting before forwarding requests to backend microservices. All inter-service communication traverses the Istio service mesh, which enforces mutual TLS encryption, identity-based access policies, and detailed telemetry collection for observability purposes.

3. Prerequisites and Requirements

Successful deployment of the CyberSOC Platform into production Kubernetes environments requires careful preparation of infrastructure resources, toolchain installation, and credential provisioning. This section enumerates all prerequisites organized by category, providing specific version requirements, configuration parameters, and verification commands that deployment teams should execute prior to initiating the installation procedure. Failure to satisfy these prerequisites may result in deployment failures, runtime instability, or security vulnerabilities that could compromise the production environment.

3.1 Infrastructure Requirements

Resource	Minimum	Recommended	Notes
Node Count	6 worker nodes	9+ worker nodes	3 AZ distribution required
vCPU Total	32 cores	64+ cores	Dedicated node pools preferred
Memory	128 GB RAM	256 GB RAM	Include headroom for bursts
Storage (SSD)	1 TB GP3/Premium SSD	2 TB+ provisioned IOPS	Separate storage classes
Network Bandwidth	10 Gbps	25+ Gbps	Cross-AZ traffic consideration
Kubernetes Version	1.28+	1.29+ (latest stable)	Upgrade path validated

Table 3.1: Infrastructure Resource Requirements

3.2 Toolchain and Software Requirements

The deployment toolchain requires specific versions of command-line utilities and cluster addons to ensure compatibility with the provided Helm charts and Kubernetes manifests. Primary tooling includes Helm 3.12 or later for chart rendering and release management, kubectl matching the cluster minor version for direct resource manipulation, and kustomize for overlay-based environment customization. Cluster-level prerequisites include installation of the cert-manager certificate lifecycle controller (v1.13+), Istio service mesh (v1.20+ with ambient mesh support optional), and the Prometheus Operator for monitoring stack management. Each prerequisite component includes health check commands in the deployment verification

section to confirm proper installation and operational status before proceeding with CyberSOC-specific resource creation.

Tool/Component	Version	Purpose	Installation Method
helm	3.12+	Package management	Official script or package manager
kubectl	matches cluster	Cluster interaction	Cloud provider CLI bundle
kustomize	5.0+	Overlay customization	kubectl apply-k or standalone binary
cert-manager	1.13+	Certificate lifecycle	Helm chart (jetstack repo)
istio/base + istiod	1.20+	Service mesh	istioctl install or Helm
prometheus-operator	0.68+	Monitoring CRDs	Helm chart (kube-prometheus-stack)
external-secrets	0.9+	Secret sync	Helm chart (external-secrets repo)

Table 3.2: Toolchain Version Requirements

4. Cluster Configuration

Production Kubernetes cluster configuration establishes the foundation upon which all CyberSOC platform components operate. This section details namespace topology, resource quotas, network policies, pod security standards, and node pool assignments that must be configured prior to deploying application workloads. The configuration parameters presented here represent hardened production defaults derived from CIS Kubernetes Benchmarks, NIST cybersecurity frameworks, and operational experience from large-scale security platform deployments.

4.1 Namespace Topology

The production deployment utilizes a multi-namespace architecture to provide logical separation between system infrastructure, application workloads, data stores, and observability components. Each namespace enforces specific resource limits, network policies, and security contexts appropriate to its contained workloads. Namespace isolation enables independent backup strategies, access control boundaries, and troubleshooting scopes that simplify operational complexity while maintaining strong security posture across the entire platform deployment.

Namespace	Purpose	Network Policy	Resource Quota
cybersoc-system	Infrastructure services	Deny-all, allow egress	cpu: 10, mem: 32Gi, pods: 50
cybersoc-workloads	Application microservices	Mesh-internal only	cpu: 40, mem: 128Gi, pods: 200
cybersoc-data	Database and cache layers	Isolated ingress	cpu: 20, mem: 96Gi, pods: 30
cybersoc-monitoring	Observability stack	Allow metrics scrape	cpu: 15, mem: 48Gi, pods: 50

Namespace	Purpose	Network Policy	Resource Quota
cybersoc-security	Security scanning, vault	Highly restricted	cpu: 5, mem: 16Gi, pods: 20

Table 4.1: Namespace Configuration Matrix

4.2 Node Pool Architecture

Production deployments leverage dedicated node pools with specialized instance types and taints/tolerations to optimize workload placement and resource utilization. Compute-optimized nodes (C-series on AWS, C2 on GCP) host CPU-intensive threat detection and SIEM correlation workloads, while memory-optimized nodes (R-series on AWS, E2/M2 on GCP) accommodate database and caching layer memory demands. Infrastructure and system components deploy to general-purpose nodes with spot instance fallback capability for cost optimization without sacrificing availability guarantees provided by PodDisruptionBudget configurations.

Node Pool	Instance Type	Autoscaling	Taint	Workload Target
system-pool	m6i.xlarge	3-6 nodes	dedicated=system:NoSchedule	Gateway, Auth, Operators
compute-pool	c6i.2xlarge	5-15 nodes	workload=compute:NoSchedule	Threat Engine, SIEM Core
memory-pool	r6i.xlarge	3-8 nodes	workload=memory:NoSchedule	PostgreSQL, Redis, ES
ui-pool	m6i.large	3-6 nodes	frontend=true:NoSchedule	Frontend, static assets
monitor-pool	m6i.large	2-4 nodes	monitoring=true:NoSchedule	Prometheus, Grafana

Table 4.2: Node Pool Specification

5. Helm Chart Structure

The CyberSOC Platform employs Helm as its primary packaging and deployment mechanism, providing templated Kubernetes manifests with configurable values for environment-specific customization. The chart architecture follows Helm best practices with a parent chart managing dependencies to subcharts for each major component group. This hierarchical structure enables independent versioning of component charts while maintaining centralized configuration management through the parent chart's values hierarchy. Deployment teams interact primarily with the parent chart, overriding values through command-line arguments, values files, or the --set flag for rapid iteration during development and staging phases.

5.1 Chart Directory Layout

```
cybersoc-platform/
├── Chart.yaml           # Parent chart metadata (version 1.0.0)
├── values.yaml          # Default production values
├── values-staging.yaml  # Staging environment overrides
├── values-dev.yaml      # Development environment overrides
├── templates/
│   ├── _helpers.tpl    # Template helper functions
│   └── _labels.tpl      # Standard label definitions
```

```

|   |   | NOTES.txt           # Post-install notes
|   |   | namespace.yaml      # Namespace creation (if needed)
|   |   | networkpolicies.yaml # Baseline network policies
|   |   | resourcequotas.yaml  # Namespace resource limits
|   |   |
|   |   | charts/
|   |   | | cybersoc-gateway/  # API Gateway subchart
|   |   | | | Chart.yaml
|   |   | | | values.yaml
|   |   | | | templates/
|   |   | | | | deployment.yaml
|   |   | | | | service.yaml
|   |   | | | | hpa.yaml
|   |   | | | | configmap.yaml
|   |   | | | | servicemonitor.yaml
|   |   | | cybersoc-auth/     # Authentication service subchart
|   |   | | cybersoc-threatengine/ # Threat detection subchart
|   |   | | cybersoc-siem/      # SIEM core subchart
|   |   | | cybersoc-database/  # Database dependencies (PostgreSQL, Redis, ES)
|   |   | | cybersoc-monitoring/ # Monitoring stack subchart
|   |   | |
|   |   | docs/
|   |   | | INSTALLATION.md    # Installation instructions
|   |   | | CONFIGURATION.md   # Values reference
|   |   | | UPGRADE.md         # Upgrade procedures

```

Figure 5.1: Complete Helm Chart Directory Structure

5.2 Parent Chart Configuration

The parent Chart.yaml defines metadata, API version compatibility, dependency declarations, and app version mappings for the entire CyberSOC platform. Dependencies are pinned to specific versions to ensure reproducible deployments, with update procedures documented separately for controlled upgrade workflows. The chart maintains Kubernetes API version compatibility annotations to enable Helm schema validation and pre-rendering checks that catch manifest errors before cluster submission.

```

apiVersion: v2
name: cybersoc-platform
description: CyberSOC AI-Native Security Operations Center Platform
type: application
version: 1.0.0
appVersion: "1.0.0"
dependencies:
- name: cybersoc-gateway
  version: "1.0.0"
  repository: "file://charts/cybersoc-gateway"
- name: cybersoc-auth
  version: "1.0.0"
  repository: "file://charts/cybersoc-auth"
- name: cybersoc-threatengine
  version: "1.0.0"
  repository: "file://charts/cybersoc-threatengine"
- name: cybersoc-siem
  version: "1.0.0"
  repository: "file://charts/cybersoc-siem"
- name: cybersoc-database
  version: "1.0.0"
  repository: "file://charts/cybersoc-database"
  condition: database.enabled
- name: cybersoc-monitoring
  version: "1.0.0"
  repository: "file://charts/cybersoc-monitoring"

```



```
    condition: monitoring.enabled
maintainers:
  - name: CyberSOC Platform Team
    email: platform@cybersoc.io
```

Listing 5.1: Parent Chart.yaml Configuration

6. Core Kubernetes Manifests

This section presents production-ready Kubernetes manifest definitions for core CyberSOC platform components. Each manifest includes comprehensive annotations for operational metadata, pod security context configurations, resource requests and limits calibrated for production workloads, probe configurations for health assessment, and service mesh integration annotations. Manifests are organized by component with explanatory commentary addressing key configuration decisions and customization points that deployment teams may need to adjust for their specific environment constraints or organizational policies.

6.1 API Gateway Deployment

The API Gateway serves as the primary ingress point for all external traffic to the CyberSOC platform, implementing authentication validation, rate limiting, request routing, and TLS termination responsibilities. The deployment configuration specifies three replicas distributed across availability zones with anti-affinity rules preventing co-location on single nodes. Resource allocations accommodate burst traffic patterns typical of security event surges during active incident response scenarios. Health probes are configured with conservative thresholds to prevent premature eviction during initialization or temporary degradation periods.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: cybersoc-gateway
  namespace: cybersoc-system
  labels:
    app.kubernetes.io/name: cybersoc-gateway
    app.kubernetes.io/component: api-gateway
    app.kubernetes.io/part-of: cybersoc-platform
    app.kubernetes.io/managed-by: Helm
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
    matchLabels:
      app.kubernetes.io/name: cybersoc-gateway
  template:
    metadata:
      labels:
        app.kubernetes.io/name: cybersoc-gateway
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/port: "9102"
        prometheus.io/path: "/metrics"
```

```

    sidecar.istio.io/inject: "true"
spec:
  serviceAccountName: cybersoc-gateway
  securityContext:
    runAsNonRoot: true
    runAsUser: 101
    fsGroup: 101
    seccompProfile:
      type: RuntimeDefault
  affinity:
    podAntiAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            labelSelector:
              matchLabels:
                app.kubernetes.io/name: cybersoc-gateway
            topologyKey: kubernetes.io/hostname
  containers:
    - name: gateway
      image: ghcr.io/cybersoc/platform-gateway:v1.0.0
      ports:
        - name: http
          containerPort: 8080
          protocol: TCP
        - name: https
          containerPort: 8443
          protocol: TCP
        - name: metrics
          containerPort: 9102
          protocol: TCP
      env:
        - name: LOG_LEVEL
          value: "info"
        - name: AUTH_SERVICE_URL
          value: "http://cybersoc-auth.cybersoc-system.svc:8080"
        - name: RATE_LIMIT_ENABLED
          value: "true"
        - name: RATE_LIMIT_REQUESTS
          value: "1000"
        - name: RATE_LIMIT_WINDOW
          value: "60"
      resources:
        requests:
          cpu: 500m
          memory: 512Mi
        limits:
          cpu: 2000m
          memory: 1Gi
      livenessProbe:
        httpGet:
          path: /healthz/live
          port: http
        initialDelaySeconds: 30
        periodSeconds: 10
        timeoutSeconds: 5
        failureThreshold: 3
      readinessProbe:
        httpGet:
          path: /healthz/ready
          port: http
        initialDelaySeconds: 10
        periodSeconds: 5

```

```

    timeoutSeconds: 3
    failureThreshold: 3
  startupProbe:
    httpGet:
      path: /healthz/startup
      port: http
    initialDelaySeconds: 10
    periodSeconds: 5
    timeoutSeconds: 3
    failureThreshold: 30
  volumeMounts:
    - name: tls-certs
      mountPath: /etc/cybersoc/tls
      readOnly: true
    - name: gateway-config
      mountPath: /etc/cybersoc/config
      readOnly: true
  volumes:
    - name: tls-certs
      secret:
        secretName: cybersoc-gateway-tls
        defaultMode: 0400
    - name: gateway-config
      configMap:
        name: cybersoc-gateway-config
        defaultMode: 0444

```

Listing 6.1: API Gateway Deployment Manifest

6.2 Gateway Service Definition

```

apiVersion: v1
kind: Service
metadata:
  name: cybersoc-gateway
  namespace: cybersoc-system
  labels:
    app.kubernetes.io/name: cybersoc-gateway
  annotations:
    service.beta.kubernetes.io/aws-load-balancer-type: nlb
    service.beta.kubernetes.io/aws-load-balancer-cross-zone-load-balancing-enabled: "true"
    service.beta.kubernetes.io/aws-load-balancer-ssl-ports: "https"
    service.beta.kubernetes.io/aws-load-balancer-ssl-cert: arn:aws:acm:region:account-id:certificate/id
spec:
  type: LoadBalancer
  externalTrafficPolicy: Local
  selector:
    app.kubernetes.io/name: cybersoc-gateway
  ports:
    - name: http
      port: 80
      targetPort: http
      protocol: TCP
    - name: https
      port: 443
      targetPort: https
      protocol: TCP

```

Listing 6.2: Gateway Service with AWS NLB Integration

6.3 Horizontal Pod Autoscaler Configuration

The Horizontal Pod Autoscaler (HPA) configuration enables automatic scaling of gateway replicas based on observed CPU utilization and custom metrics from request throughput. The HPA targets 70% average CPU utilization as the primary scaling trigger with a minimum replica count of 3 (for availability guarantees) and maximum of 20 replicas to handle extreme traffic bursts during security incidents. Custom metrics integration with Prometheus Adapter allows scaling decisions based on requests-per-second and queue depth metrics that better represent actual load than CPU alone.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: cybersoc-gateway-hpa
  namespace: cybersoc-system
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: cybersoc-gateway
  minReplicas: 3
  maxReplicas: 20
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300
      policies:
        - type: Percent
          value: 10
          periodSeconds: 60
    scaleUp:
      stabilizationWindowSeconds: 0
      policies:
        - type: Percent
          value: 100
          periodSeconds: 15
        - type: Pods
          value: 4
          periodSeconds: 15
      selectPolicy: Max
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Pods
      pods:
        metric:
          name: requests_per_second
        target:
          type: AverageValue
          averageValue: "1000"
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 80
```

Listing 6.3: Gateway HPA with Multi-Metric Scaling

7. Database Deployment

The CyberSOC Platform requires persistent data stores for transactional data, caching layers, and log aggregation. This section provides deployment configurations for PostgreSQL (primary relational database), Redis (in-memory cache and message broker), and Elasticsearch (log storage and full-text search). Database deployments utilize StatefulSets for ordered deployment, stable network identifiers, and persistent volume claims with appropriate storage classes for production performance requirements. Backup and recovery procedures are integrated through sidecar containers and CronJob schedules documented in the Disaster Recovery section.

7.1 PostgreSQL StatefulSet

PostgreSQL deployment uses the CloudNativePG operator for production-grade PostgreSQL management, providing automated failover, backup scheduling, and connection pooling capabilities. The operator manages Patroni-based high availability clusters with streaming replication between primary and replica instances. Storage configuration utilizes provisioned IOPS SSD volumes with 3000 IOPS baseline to support the concurrent query patterns generated by SIEM correlation queries and audit trail writes. Connection pooling via PgBouncer sidecar reduces connection overhead for stateless application pods that frequently connect and disconnect.

```
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: cybersoc-postgres
  namespace: cybersoc-data
spec:
  instances: 3
  imageName: ghcr.io/cloudnative-pg/postgresql:16.2
  primaryUpdateStrategy: unsupervised
  postgresql:
    parameters:
      max_connections: "200"
      shared_buffers: "1GB"
      effective_cache_size: "3GB"
      maintenance_work_mem: "256MB"
      wal_compression: "on"
      max_wal_senders: "3"
      max_replication_slots: "3"
      hot_standby: "on"
      log_min_duration_statement: "1000"
  pg_hba:
    - hostssl all all 0.0.0.0/0 scram-sha-256
    - host all all 127.0.0.1/32 md5
  bootstrap:
    initdb:
      database: cybersoc
      owner: cybersoc_admin
      secret: cybersoc-postgres-admin
      encoding: UTF8
      locale: C.UTF-8
      dataChecksums: enabled
  storage:
    storageClass: gp3-crypted
    size: 500Gi
```

```

    resizeInUseVolumes: true
resources:
  requests:
    cpu: 1000m
    memory: 4Gi
  limits:
    cpu: 2000m
    memory: 8Gi
monitoredServices:
  - type: prometheus
backup:
  retentionDays: 30
  barmanObjectStore:
    destinationPath: s3://cybersoc-backups/postgresql
    endpointUrl: https://s3.amazonaws.com
    s3Credentials:
      accessKeyId:
        name: cybersoc-backup-creds
        key: ACCESS_KEY_ID
      secretAccessKey:
        name: cybersoc-backup-creds
        key: SECRET_ACCESS_KEY
    serverName: cybersoc-postgres-current
    wal:
      compression: gzip
      maxParallel: 2
affinity:
  topologySpreadConstraints:
    - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: DoNotSchedule
    labelSelector:
      matchLabels:
        postgresql.cnpg.io/cluster: cybersoc-postgres

```

Listing 7.1: CloudNativePG Cluster Definition

7.2 Redis Cluster Configuration

Redis deployment operates in cluster mode with six nodes (three masters, three replicas) providing high availability and horizontal partitioning for dataset sizes exceeding single-node memory capacity. The cluster handles session state storage, real-time event pub/sub messaging, API rate limiting counters, and frequently-accessed configuration caching. Authentication is enforced via ACL rules with dedicated users for each application service, following least-privilege principles. Persistence configuration enables AOF (Append-Only File) with fsync-every-second for durability acceptable trade-off versus pure in-memory performance.

```

apiVersion: redis.redis.opstreelabs.in/v1beta1
kind: RedisCluster
metadata:
  name: cybersoc-redis
  namespace: cybersoc-data
spec:
  clusterSize: 6
  redisImage: redis:7.2-alpine
  redisConfig:
    maxmemory-policy: allkeys-lru
    save: ""
    appendonly: "yes"
    appendfsync: "everysec"
    tcp-keepalive: "300"

```

```

    timeout: "0"
    aclfile: /etc/redis/user.acl
    persistenceEnabled: true
    storage:
      volumeClaimTemplate:
        spec:
          storageClassName: gp3-crypted
          resources:
            requests:
              storage: 50Gi
    resources:
      requests:
        cpu: 500m
        memory: 1Gi
      limits:
        cpu: 1000m
        memory: 2Gi
    podSecurityContext:
      fsGroup: 1000
      runAsNonRoot: true
      runAsUser: 999
    nodeSelector:
      workload: memory
    tolerations:
      - key: workload=memory
        operator: Exists
        effect: NoSchedule
    topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: kubernetes.io/zone
        whenUnsatisfiable: DoNotSchedule
        labelSelector:
          matchLabels:
            app.redis.opstreelabs.in/name: cybersoc-redis

```

Listing 7.2: Redis Cluster with Persistence

8. Service Mesh Integration

Istio service mesh integration provides zero-trust networking for all CyberSOC platform components, enforcing mutual TLS encryption for all inter-service communication regardless of network perimeter security assumptions. The mesh configuration defines authorization policies implementing intent-based access control, telemetry collection for distributed tracing and metrics, and traffic management rules for canary deployments and circuit breaking. This section documents the istiod control plane configuration, sidecar injection policies, and security-focused mesh settings appropriate for handling sensitive security operations data.

8.1 Mesh-Wide mTLS Configuration

PeerAuthentication resources define mTLS enforcement modes at namespace and workload granularity. The production configuration sets STRICT mode for all CyberSOC namespaces, requiring mutual TLS for all inbound connections and rejecting unencrypted traffic. Certificate management leverages Istio's built-in Citadel (now integrated into istiod) for automatic X.509 certificate issuance and rotation with 24-hour certificate validity and 1-hour rotation advance window. Workload identities integrate with Kubernetes ServiceAccount tokens via

SPIFFE/SPIRE-compliant identity documents enabling cross-cluster mesh federation if multi-cluster deployment is implemented.

```
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: default-mtls-strict
  namespace: cybersoc-system
spec:
  mtls:
    mode: STRICT
---
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: default-mtls-strict
  namespace: cybersoc-workloads
spec:
  mtls:
    mode: STRICT
---
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: deny-all-default
  namespace: cybersoc-workloads
spec:
  {}
  action: DENY
---
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: allow-internal-traffic
  namespace: cybersoc-workloads
spec:
  action: ALLOW
  rules:
    - from:
        - source:
            principals:
              - "cluster.local/ns/cybersoc-system/sa/cybersoc-gateway"
              - "cluster.local/ns/cybersoc-system/sa/cybersoc-auth"
        - source:
            namespaces:
              - "cybersoc-workloads"
      to:
        - operation:
            methods: ["GET", "POST", "PUT", "DELETE", "PATCH"]
            notPaths: ["/admin/*", "/debug/*"]
```

Listing 8.1: Istio mTLS and Authorization Policies

8.2 Destination Rules and Traffic Management

DestinationRule configurations establish connection pool settings, circuit breaker thresholds, and load balancing algorithms for service-to-service communication within the mesh. These settings protect backend services from cascading failures during traffic spikes or degraded downstream dependencies. Load balancing utilizes locality-aware round-robin to prefer same-zone endpoints, reducing cross-zone network costs and latency while maintaining

distribution across availability zones for fault tolerance.

```
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: cybersoc-threat-engine
  namespace: cybersoc-workloads
spec:
  host: cybersoc-threat-engine
  trafficPolicy:
    loadBalancer:
      simple: LEAST_REQUEST
      localityLbSetting:
        enabled: true
        distribute:
          - from: us-east-1/us-east-1a
            to:
              "us-east-1/us-east-1a": 80
              "us-east-1/us-east-1b": 10
              "us-east-1/us-east-1c": 10
    connectionPool:
      tcp:
        maxConnections: 100
        connectTimeout: 5s
        tcpKeepalive:
          time: 7200s
          interval: 75s
      http:
        h2UpgradePolicy: DEFAULT
        http1MaxPendingRequests: 100
        http2MaxRequests: 1000
        maxRequestsPerConnection: 10
        maxRetries: 3
    outlierDetection:
      consecutive5xxErrors: 5
      interval: 30s
      baseEjectionTime: 60s
      maxEjectionPercent: 50
      minHealthPercent: 51
  tls:
    mode: ISTIO_MUTUAL
```

Listing 8.2: Threat Engine Destination Rule with Circuit Breaking

9. Security Hardening

Security hardening configurations implement defense-in-depth protections aligned with CIS Kubernetes Benchmark, NIST SP 800-204B (Zero Trust Architecture), and PCI-DSS requirements applicable to security operations platforms. This section addresses pod security standards, network policy enforcement, secrets management integration, container image integrity verification, and runtime security monitoring. All configurations represent mandatory production requirements; deviations require formal risk acceptance documentation and compensating control implementation approved by the CyberSOC security architecture team.

9.1 Pod Security Standards

Kubernetes Pod Security Standards (PSS) enforce baseline, restricted, and privileged policy levels at the namespace level through built-in admission controllers or third-party policy engines such as Kyverno or OPA Gatekeeper. CyberSOC production namespaces enforce the Restricted profile as minimum, prohibiting privilege escalation, requiring non-root execution, dropping all capabilities by default, and mandating read-only root filesystems. Exceptions for specific workloads requiring elevated privileges (such as node-agent based security scanners) undergo enhanced review and are deployed in isolated namespaces with Privileged profile explicitly acknowledged.

```
apiVersion: psp.policy/v1alpha1
kind: PodSecurityPolicy
metadata:
  name: cybersoc-restricted
  annotations:
    seccomp.security.alpha.kubernetes.io/allowedProfiles: runtime/default
    apparmor.security.alpha.kubernetes.io/allowedProfiles: runtime/default
spec:
  privileged: false
  allowPrivilegeEscalation: false
  requiredDropCapabilities:
    - ALL
  volumes:
    - 'configMap'
    - 'emptyDir'
    - 'projected'
    - 'secret'
    - 'downwardAPI'
    - 'persistentVolumeClaim'
  hostNetwork: false
  hostIPC: false
  hostPID: false
  runAsUser:
    rule: MustRunAsNonRoot
  runAsGroup:
    rule: MustRunAs
    ranges:
      - min: 100
        max: 65535
  supplementalGroups:
    rule: MustRunAs
    ranges:
      - min: 100
        max: 65535
  fsGroup:
    rule: MustRunAs
    ranges:
      - min: 100
        max: 65535
  readOnlyRootFilesystem: true
  seccompProfile:
    type: RuntimeDefault
```

Listing 9.1: Restricted Pod Security Policy

9.2 Network Policy Implementation

Network policies implement zero-trust networking principles by default-deny-ingress at the namespace level, then selectively permitting required traffic flows through explicitly defined policy rules. The policy architecture follows a layered approach with baseline namespace

policies establishing deny-all defaults, followed by component-specific policies authorizing necessary communication patterns. Policy enforcement occurs at the kernel level through kube-proxy iptables/ipvs rules or, when using CNI plugins like Calico or Cilium, through more efficient eBPF-based implementations supporting L7 policy awareness.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all-ingress
  namespace: cybersoc-workloads
spec:
  podSelector: {}
  policyTypes:
  - Ingress
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-gateway-to-workloads
  namespace: cybersoc-workloads
spec:
  podSelector:
    matchLabels:
      app.kubernetes.io/part-of: cybersoc-platform
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: cybersoc-system
      podSelector:
        matchLabels:
          app.kubernetes.io/name: cybersoc-gateway
    ports:
    - protocol: TCP
      port: 8080
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-same-namespace
  namespace: cybersoc-workloads
spec:
  podSelector:
    matchLabels:
      app.kubernetes.io/part-of: cybersoc-platform
  policyTypes:
  - Ingress
  - Egress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app.kubernetes.io/part-of: cybersoc-platform
  egress:
  - to:
    - podSelector:
        matchLabels:
          app.kubernetes.io/part-of: cybersoc-platform
```

Listing 9.2: Zero-Trust Network Policies

10. Deployment Procedures

This section provides step-by-step deployment procedures for installing the CyberSOC Platform into production Kubernetes environments. Procedures are sequenced in dependency order, beginning with cluster-level prerequisites and progressing through infrastructure components, databases, application services, and finally post-deployment validation. Each procedure includes pre-flight checks, execution commands, expected outputs, and rollback triggers. Automated deployment through CI/CD pipelines should incorporate these procedures as referenced job definitions with appropriate approval gates for production environments.

10.1 Pre-Flight Verification Checklist

Before initiating deployment, execute the following verification commands to confirm cluster readiness and prerequisite satisfaction. Any check failure must be resolved before proceeding; attempting deployment against an unprepared cluster will result in undefined behavior, partial installations requiring manual cleanup, or runtime failures affecting platform stability. Document all pre-flight results in the deployment runbook for post-incident analysis if issues arise during subsequent steps.

Check	Command	Expected Output	Remediation
K8s Version	kubectl version --short	v1.28+ server	Upgrade cluster
Node Readiness	kubectl get nodes	Ready status	Investigate nodes
Helm Installed	helm version	v3.12+	Install/upgrade Helm
Cert Manager	kubectl get pods -n cert-manager	Running	Install cert-manager
Istiod Running	kubectl get pods -n istio-system	Running	Install Istio
Storage Classes	kubectl get sc	gp3-crypted exists	Create SC
Quota Available	kubectl resourcequota -n cybersoc	HardLimit > 50%	Adjust quotas

Table 10.1: Pre-Flight Verification Checklist

10.2 Step-by-Step Deployment Workflow

Step 1: Create Namespaces and Base Resources Execute namespace creation and apply resource quotas, network policies, and PodSecurityPolicy bindings that establish the operational boundaries for all subsequent deployments. Verify namespace creation before proceeding.

```
kubectl apply -f k8s/base/namespaces.yaml
kubectl apply -f k8s/base/resourcequotas.yaml
kubectl apply -f k8s/base/networkpolicies.yaml
```

kubectl get namespaces | grep cybersoc Step 2: Install Certificates and Secrets Deploy certificate issuers (Let's Encrypt production or enterprise CA), request TLS certificates for gateway and internal services, and configure ExternalSecret resources to synchronize credentials from HashiCorp Vault or cloud KMS into Kubernetes secrets. helm install cert-manager jetstack/cert-manager -n cert-manager

```

kubect1 apply -f k8s/secrets/certificate-issuers.yaml
kubect1 apply -f k8s/secrets/external-secrets.yaml
kubect1 wait --for=condition=ready certificate/cybersoc-gateway-tls -n cybersoc-system --timeout=300s Step 3:
Deploy Database Layer Initialize PostgreSQL cluster via CloudNativePG operator, Redis cluster
via Redis operator, and Elasticsearch cluster via ECK operator. Wait for all database clusters to
reach Ready state before proceeding to application deployment. helm dependency build
charts/cybersoc-database/
helm install cybersoc-database ./charts/cybersoc-database -n cybersoc-data -f values-prod-db.yaml
kubect1 wait --for=condition=Ready cluster/cybersoc-postgres -n cybersoc-data --timeout=600s
kubect1 wait --for=condition=Ready rediscluster/cybersoc-redis -n cybersoc-data --timeout=300s Step 4: Deploy
Application Services Install the main CyberSOC platform chart with production values file,
triggering deployment of gateway, authentication, threat engine, SIEM core, case management,
and UI frontend components. Monitor rollout progress for each component. helm dependency build ./
helm install cybersoc-platform ./ -n cybersoc-workloads -f values-production.yaml --timeout 15m
kubect1 rollout status deployment/cybersoc-gateway -n cybersoc-system --timeout=300s
kubect1 rollout status deployment/cybersoc-threat-engine -n cybersoc-workloads --timeout=600s Step 5: Configure
DNS and Ingress Create DNS records pointing to the gateway load balancer, configure
VirtualService routes for path-based routing, and verify end-to-end TLS connectivity from
external clients through to backend services. export LB_HOST=$(kubect1 get svc cybersoc-gateway -n
cybersoc-system -o jsonpath='{.status.loadBalancer.ingress[0].hostname}')
# Update DNS: api.cybersoc.example.com -> $LB_HOST
kubect1 apply -f k8s/networking/virtualservices.yaml
curl -vvv https://api.cybersoc.example.com/healthz Step 6: Post-Deployment Validation Execute
comprehensive health checks, smoke tests, and integration verification to confirm all
components are operational and communicating correctly. Document validation results and any
deviations requiring follow-up. ./scripts/validate-deployment.sh --environment production
./scripts/smoke-tests.sh --endpoint https://api.cybersoc.example.com
kubect1 top nodes && kubect1 top pods -A | sort -k3 -h

```

11. Scaling Strategies

Production deployment must accommodate variable workloads ranging from steady-state operation during normal business hours through extreme demand spikes during active security incidents affecting multiple customers simultaneously. This section documents horizontal and vertical scaling configurations, auto-scaling trigger calibration, and operational procedures for manual scaling interventions when automated responses prove insufficient. Scaling strategies balance responsiveness to demand changes against cost efficiency and stability during scale events.

11.1 Horizontal Pod Autoscaler Tuning

HPA tuning requires understanding of workload characteristics to select appropriate metrics, threshold values, and stabilization windows that prevent oscillation (replica count thrashing between scale-up and scale-down events). CPU-based scaling provides baseline responsiveness

but poorly represents actual capacity pressure for I/O-bound or queue-driven workloads. Custom metrics from Prometheus (requests per second, queue depth, latency percentiles) enable more accurate scaling decisions aligned with user experience quality indicators rather than infrastructure utilization proxies.

Component	Min Replicas	Max Replicas	Primary Metric	Target	Scale-up Rate
API Gateway	3	20	CPU + RPS	70% / 1000 req/s	100% per 15s
Threat Engine	5	50	Queue Depth	< 100 events	4 pods per 15s
SIEM Core	3	25	CPU + Latency P99	75% / < 500ms	100% per 30s
Case Management	2	10	CPU	70%	50% per 60s
Frontend	3	15	CPU + Connections	70% / 5000 conn	100% per 15s

Table 11.1: HPA Configuration by Component

11.2 Cluster Autoscaler Configuration

When pod scaling exhausts available node capacity, the Cluster Autoscaler (or cloud-provider equivalent such as Karpenter for AWS or Autopilot for GKE) provisions new nodes to accommodate pending pods. Configuration balances over-provisioning waste against startup latency for cold nodes. Scale-down thresholds prevent premature node termination that would trigger disruptive pod rescheduling. Priority-based scoring ensures critical system components receive node capacity before lower-priority batch workloads during resource-constrained situations.

12. Monitoring and Observability

Comprehensive monitoring and observability are essential for operating the CyberSOC Platform at production scale, enabling rapid detection of anomalies, efficient root cause analysis during incidents, and data-driven capacity planning. The monitoring stack integrates Prometheus for metrics collection and alerting, Grafana for dashboard visualization, Jaeger for distributed tracing, and Loki for log aggregation. This section documents the monitoring architecture, key metrics to track, alert threshold configurations, and dashboard layouts for operational visibility.

12.1 Key Performance Indicators

KPI Category	Metric Name	Alert Threshold	Dashboard Panel
Availability	Uptime Percentage	< 99.9% triggers P1	SLA Overview
Latency	API Response P50	> 200ms warning	Latency Distribution
Latency	API Response P99	> 2000ms critical	Latency Distribution
Throughput	Events Processed/sec	< baseline - 30%	Throughput Gauge

KPI Category	Metric Name	Alert Threshold	Dashboard Panel
Error Rate	HTTP 5xx Errors	> 1% warning, > 5% critical	Error Budget
Saturation	CPU Utilization	> 80% sustained	Resource Utilization
Saturation	Memory Utilization	> 85% warning	Resource Utilization
Internal	Queue Depth	> 1000 events	Pipeline Status
Business	Active Incidents	> 10x baseline	Operations Overview

Table 12.1: Critical KPIs and Alert Thresholds

13. Disaster Recovery

Disaster recovery planning ensures the CyberSOC Platform can recover from catastrophic failures including entire availability zone loss, region-wide outages, or data corruption events. This section documents backup strategies for each data tier, recovery time objectives (RTO) and recovery point objectives (RPO) by component, failover procedures, and testing cadence requirements. DR procedures must be practiced regularly through chaos engineering exercises to validate assumptions and identify gaps before real incidents expose deficiencies.

13.1 Backup Strategy by Data Tier

Data Tier	Technology	Backup Frequency	Retention	RPO Target
PostgreSQL	CloudNativePG WAL + Base	Continuous WAL, Daily Base	30 days	< 5 min
Redis AOF	S3 Snapshot	Hourly	7 days	< 1 hour
Elasticsearch	Snapshot Repository	Every 6 hours	14 days	< 6 hours
Object Storage (S3)	Cross-Region Replication	Real-time CRR	90 days	Near-zero
Secrets (Vault)	Raft Auto-snapshot	Automatic	30 days	< 15 min
GitOps Repo	Geo-redundant Git	On push	Indefinite	< 1 min

Table 13.1: Backup Strategy Matrix

13.2 Failover Procedure

In the event of primary region failure, initiate the documented failover procedure to activate the standby deployment in the secondary region. The procedure assumes standby infrastructure is pre-provisioned and continuously replicating data from primary; cold-start provisioning is not covered as it exceeds RTO targets. Failover execution requires coordination between platform team (infrastructure failover), security team (access restoration validation), and customer success team (communication regarding service interruption). Failover Steps: 1. Confirm primary region outage via multiple monitoring sources (avoid false positive triggers) 2. Execute

DNS cutover to secondary region load balancer (TTL should be 60 seconds maximum) 3. Activate read-write mode on standby PostgreSQL cluster (promote replica) 4. Redirect object storage traffic to secondary bucket 5. Validate end-to-end functionality via smoke test suite 6. Communicate status to stakeholders via established notification channels 7. Initiate post-mortem timeline reconstruction once primary region recovers

14. Troubleshooting Guide

This troubleshooting guide addresses common operational issues encountered during CyberSOC Platform deployment and operation. Each issue entry includes symptom description, diagnostic commands, root cause analysis guidance, and resolution steps. For issues not covered in this guide, escalate to the platform engineering team via the designated PagerDuty rotation with collected diagnostic artifacts including pod logs, events, and metrics snapshots surrounding the issue timeframe.

14.1 Common Issues and Resolutions

Symptom	Likely Cause	Diagnostic Command	Resolution
Pod CrashLoopBackOff	Missing ConfigMap/Secret	kubectl describe pod	Verify resource existence
Pod Pending (InsufficientCPU)	Cluster at capacity	kubectl describe node	Scale cluster or reduce requests
503 from Gateway	Backend unhealthy	kubectl get endpoints	Check backend pod health
TLS Handshake Failure	Certificate expired/missing	kubectl get certificate	Renew via cert-manager
High Memory OOMKills	Limits too low	kubectl top pods	Increase memory limits
Slow API Responses	Database connection exhaustion	check DB pool metrics	Increase pool size
Intermittent 500 Errors	Pod being evicted (PDB)	kubectl get pdb	Adjust PDB or add capacity
Metrics Missing	ServiceMonitor misconfigured	kubectl get servicemonitor	Fix label selectors

Table 14.1: Common Issues Quick Reference